Improving the Pretraining Efficiency of Diffusion Language Models

Chang-Han Chen
University of California, Berkeley
Berkeley, CA 94720, USA
changhanc@berkeley.edu

Abstract

Diffusion language models (dLMs) are slow to train but can eventually surpass autoregressive (AR) models in the data-constrained setting. Can we accelerate the crossover point at which dLMs win? We already found some interesting results about curriculum pretraining at 1M scale on TinyShakespeare and 50M scale on NVIDIA's ClimbMix, and outline upcoming data-constrained scaling experiments at 50M–300M. We suspect that the curriculum for optimal perplexity would be mostly AR, but the curriculum for optimal downstream performance would be more diffusion-heavy.

1 Introduction

The slogan is that "Diffusion models are super data learners" Ni et al. (2025): when unique training data is limited, the optimal performance of diffusion language models (dLMs) on downstream tasks can surpass autoregressive (AR) models. For example, a 1B-parameter dLM achieves over 56% on HellaSwag using only 1B unique tokens. In fact, we are very likely moving toward an era where pretraining is more constrained by data than compute.

But the reality is that dLMs remain substantially less compute-efficient to pretrain than AR models. Masked diffusion (MDLM) is roughly $14\times$ less efficient than AR per training token; uniform-noise diffusion is even worse, roughly $32\times$ less efficient. This is intuitive because any-order progressive decoding is a strictly harder task than autoregressive left-to-right generation.

If one really believes that dLMs are super data learners, then the top, if not only, priority should be to increase their compute efficiency. There have been numerous works on A2D conversion (Fu et al., 2025; Tian et al., 2025; Bie et al., 2025; Cheng et al., 2025). However, they all start with off-the-shelf, pretrained LLMs such as LLaMA or Qwen. Although this is convenient for getting strong benchmark results for the final model, it does not address the fundamental problem of how to improve the efficiency of dLM pretraining.

In this work, we study A2D conversion as a pretraining paradigm from scratch, not (necessarily) as continual pretraining from off-the-shelf LLMs. In fact, the traditional A2D conversion becomes much more principled from this perspective: it is a special case of the *curriculum pretraining* studied in our work, where the majority of the curriculum is next-token prediction.

This work is largely still in progress, mostly due to compute limitations. But we still present some interesting results: 1M-scale models on TinyShakespeare and a 50M model on NVIDIA's ClimbMix.

2 Pilot Study: TinyShakespeare

We first validate the approach at small scale. All models share a transformer backbone with RoPE, QK-norm, ReGLU MLPs, and RMSNorm, spanning six sizes from 0.1M to 3M parameters. We compare four diffusion objectives—MDLM, Remasked, Edit-1, and Edit-2—each in standard and block (block length 4) formulations. We fix an IsoFLOP budget of $C = 6 \times 10^{14}$ and determine training steps from the budget, model size, and per-step FLOP multiplier ($6\times$ for standard, $12\times$ for block, $24\times$ for block Edit-2). The details of these variants are not important because we find that they are not worth scaling up.

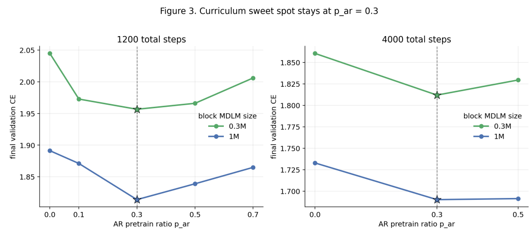

Figure 1: 30% AR warmup is the most effective even when compute becomes 3x (1200 to 4000 steps).

Table 1: Pilot: validation CE for AR→Block MDLM curriculum on TinyShakespeare. Bold = best per row.

	1200 total steps					
Size	p=0	p=0.1	p=0.3	p=0.5	p=0.7	Δ
0.3M	2.045	1.973	**1.957**	1.966	2.006	−0.088
1M	1.891	1.871	**1.814**	1.839	1.865	−0.077
	4000 total steps					
Size	p=0	p=0.3	p=0.5			Δ
0.3M	1.860	**1.812**	1.830			−0.049
1M	1.733	**1.690**	1.692			−0.043

AR warm-start has a U-shaped optimum at $p_{\mathrm{AR}} = 0.3$. We train AR for a fraction p_{AR} of total steps, then switch to block MDLM. Table 1 shows that $p_{\mathrm{AR}} = 0.3$ consistently gives the lowest validation loss: -0.088 nats at 0.3M and -0.077 nats at 1M. Too little AR undertrains the initialization; too much starves the diffusion phase of training steps (Figure 1).

3 Scaling to ClimbMix

We now ask whether the pilot findings hold at realistic scale: BPE tokenization, web-text data, and models large enough for meaningful downstream evaluation.

3.1 Setup

Data and model. We train on ClimbMix-400B (BPE, vocab 8192, seq len 2048) with a 50M-parameter transformer (d=768, 7 layers, 12 heads, $N \approx$ 49.5M non-embedding params) using RoPE, QK-norm, ReGLU MLPs, and RMSNorm. Three model families share the same backbone: AR, MDLM (masked diffusion), and BD3-LM (block diffusion, `block_len`=16). All runs use the NorMuon+AdamW grouped optimizer (AdamW on embeddings and scalar parameters, NorMuon on matrix parameters) with a warmup-stable LR schedule (5% linear warmup, then constant).

Curricula. *C0 (plain):* AR → BD3(16), sweep $p_{\mathrm{AR}} \in \{0.2, 0.3, 0.5, 0.8\}$. *C1 (geometric):* AR → BD3(2) → BD3(4) → BD3(8) → BD3(16), 20% steps each. Both are compared against pure baselines (AR-only, MDLM-only, BD3-only).

3.2 Curriculum Results

All curriculum runs use the NorMuon+AdamW optimizer at 50M with 1000 total steps. The AR warm-up phase (200 steps for C1 and C2, variable for C0) is trained once and shared across variants that branch from the same AR checkpoint.

C0 (plain). Table 2 shows C0 results. $p_{\mathrm{AR}} = 0.8$ (800 AR steps, 200 BD3 steps) reaches train loss 3.949, substantially ahead of $p_{\mathrm{AR}} = 0.2$ and $p_{\mathrm{AR}} = 0.3$, whose BD3 phases were interrupted before completion. This contrasts with TinyShakespeare, where $p_{\mathrm{AR}} = 0.3$ was optimal.

Table 2: C0 curriculum, NorMuon+AdamW, 50M / 1000 total steps. BD3 train loss at last observed step.

Config	AR steps	BD3 progress	Train loss	Status
p_{AR}=0.8	800	200 / 200	**3.949**	done
p_{AR}=0.3	300	400 / 700	4.419	interrupted
p_{AR}=0.2	200	500 / 800	4.416	interrupted

C1 and C2 (multi-stage). Both C1 and C2 use 200 AR steps followed by four BD3 stages of 200 steps each. Table 3 reports the train loss at the end of each stage. C1 (geometric doubling) reaches 4.282 and slightly outperforms C2 (aggressive jump) at 4.322. Both are substantially behind C0 with $p_{\mathrm{AR}}=0.8$ (3.949), confirming that at 50M scale and 1000 total steps, more AR warmup beats multi-stage diffusion ramps on train loss.

Table 3: Per-stage train loss for C1 and C2 curricula (NorMuon+AdamW, 50M, 200 steps/stage). Each stage warm-starts from the previous stage's checkpoint.

Curriculum	Stage	Start loss	End loss	End grad norm
C1 (geometric)	BD3(2)	—	—	—
	BD3(4)	—	—	—
	BD3(8)	—	—	—
	BD3(16)	4.332	**4.282**	0.291
C2 (aggressive)	BD3(8)	7.806	4.500	0.270
	BD3(64)	4.910	4.567	0.261
	BD3(512)	5.038	4.578	0.240
	BD3(16)	4.559	**4.322**	0.239

Several patterns emerge from the C2 trace. The BD3(8) stage does the heavy lifting, dropping loss from 7.81 to 4.50. The subsequent BD3(64) and BD3(512) stages make little progress and in fact regress slightly from where BD3(8) ended, suggesting that very large block lengths do not help at this scale. The final BD3(16) stage recovers and provides a modest additional improvement.

4 NEXT STEP

Curriculum 3 (AR-heavy warmup): AR $\rightarrow$ BD3(4) $\rightarrow$ BD3(16) with a swept AR fraction (20–50%), testing whether a two-stage diffusion ramp-up outperforms the single-jump C0. This is the remaining curriculum to run before selecting the best design for the scaling experiments below.

Data-constrained scaling laws The starting point of the project was the Super Learner paper (Ni et al., 2025), as explained in the introduction. It would be meaningless if our curricula make dLMs more compute-efficient, so closer to standard AR, but at the same time deprive them of their superior downstream performance in the data-constrained setting.

To directly test the thesis that our curricula accelerate not only the early progress of dLMs, but also, more importantly, the crossover point at which dLMs surpass AR, a minimal experiment setup is as follows.

We fix the total training budget at 500M tokens and vary the unique-data pool ($U \in \{25\text{M}, 50\text{M}, 100\text{M}\}$), comparing pure AR, pure BD3, and the best curriculum on HellaSwag accuracy. The key metric is the *crossover step*: the first checkpoint at which a non-AR method exceeds AR by a meaningful margin (≥ 0.5 absolute HellaSwag points for ≥ 2 consecutive checkpoints). If the curriculum shifts this crossover earlier, it confirms that the AR warm-start improves not just training loss but the practical data-efficiency advantage of diffusion in the multi-epoch regime.

It would be very interesting if it turns out that the scaling law of validation loss suggests the majority of the curriculum should be AR, but that of HellaSwag performance suggests otherwise. Based on our preliminary results, this seems very plausible.

5 Application to VESSL Credits

I intend to use approximately 1,250 A100 hours to scale up data-constrained scaling laws inSection 4. All experiments use the NorMuon+AdamW grouped optimizer exclusively (AdamW on embeddings and scalar parameters, NorMuon on matrix parameters), based on Phase 1 calibration results (Section 3.2). The primary investment is the data-constrained crossover experiment at three model scales: 50M, 100M, and 300M parameters. Per-run compute scales roughly 2$\times$ from 50M to 100M and 6$\times$ from 50M to 300M. Table 4 breaks down the estimated compute budget.

Table 4: Estimated A100-hour budget for proposed experiments (NorMuon+AdamW optimizer only).

Experiment	Hours	Notes
A. Curriculum search (50M)	20	C0–C3 + baselines, NorMuon only, 1000 steps each
B. Data-constrained crossover (50M)	105	3 methods $\times$ 3 U values $\times$ 3 seeds; post-hoc HellaSwag eval
C. Data-constrained crossover (100M)	210	Same scope as B at 2$\times$ compute per run
D. Data-constrained crossover (300M)	630	Same scope as B at 6$\times$ compute per run
Subtotal	**965**	
Re-runs and debugging ($\sim$30%)	290	
Grand total	**$\sim$1,255**	

Although our current setup deliberately uses a minimal model to isolate the effect of curriculum design, we are actively investigating modern architectural and training interventions that improve compute efficiency. Inspired by IMU-1 (Grigorev, 2026), significant gains can come from carefully combining known techniques—value embeddings, layer-norm rescaling, GQA, among others—and we plan to add a dedicated ablation section for these interventions on dLMs if compute allows. We are also exploring Mixture-of-Experts architectures based on OLMoE (Muennighoff et al., 2024), and following a suggestion by Sewon Min in private conversation, we may allocate part of the budget to MoE-related experiments.

Acknowledgments

Experiments were run on a single NVIDIA A100-SXM4-80GB GPU rented on Vast.ai.

References

Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, et al. LLaDA 2.0: Scaling up diffusion language models to 100b. *arXiv preprint arXiv:2512.15745*, 2025.

Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, and Qian Yao. SDAR: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025.

Yonggan Fu, Lexington Whalen, Zhifan Ye, Xin Dong, Shizhe Diao, and Song Han. Efficient-DLM: From autoregressive to diffusion language models, and beyond in speed. *arXiv preprint arXiv:2512.14067*, 2025.

George Grigorev. IMU-1: Sample-efficient pre-training of small language models, 2026.

Niklas Muennighoff, Luca Tang, Loubna Ben Allal, et al. OLMoE: Open mixture-of-experts language models. *arXiv preprint arXiv:2409.02060*, 2024.

Jinjie Ni, Qian Liu, Longxu Dou, Chao Du, Zili Wang, Hang Yan, Tianyu Pang, and Michael Qizhe Shieh. Diffusion language models are super data learners. *arXiv preprint arXiv:2511.03276*, 2025.

Yuchuan Tian, Yuchen Liang, Shuo Zhang, Yingte Shu, and Guangwen Yang. From next-token to next-block: A principled adaptation path for diffusion LLMs. *arXiv preprint arXiv:2512.06776*, 2025.